Pelo que vejo no seu print, o Git está configurado com o seu e-mail, mas a conexão SSH com o GitHub ainda **não funcionou**.

Note a última linha no terminal:

```
git@github.com: Permission denied (publickey).
```

Isso significa que, embora o Git Bash "conheça" o GitHub, ele não conseguiu se autenticar porque a sua **chave pública SSH** ainda não foi configurada ou adicionada à sua conta do GitHub. Sem isso, você não conseguirá enviar (push) ou baixar (pull) código de repositórios privados.

Como resolver isso (Passo a Passo)

Para que o VS Code e o Git Bash funcionem 100% com o GitHub, siga estes passos:

1. Verifique se você já tem uma chave

Digite o comando abaixo no Git Bash:

Bash

```bash
ls -al ~/.ssh
```

Se não aparecer nada como `id_ed25519.pub` ou `id_rsa.pub`, você precisa criar uma.

2. Crie uma nova chave SSH

Se não tiver, digite (substituindo pelo seu e-mail):

Bash

```bash
ssh-keygen -t ed25519 -C "eliandro_avelar@yahoo.com.br"
```

- Aperte **Enter** para todas as perguntas (não precisa de senha, a menos que queira segurança extra).

3. Copie a chave gerada

Agora, mostre o conteúdo da chave no terminal para copiá-lo:

Bash

```
cat ~/.ssh/id_ed25519.pub
```

- Selecione todo o texto que começar com `ssh-ed25519` e termine com seu e-mail, e **copie**.

4. Adicione ao GitHub

1. Vá ao seu GitHub no navegador.
2. Clique na sua foto de perfil (canto superior direito) > **Settings**.
3. No menu lateral esquerdo, clique em **SSH and GPG keys**.
4. Clique no botão verde **New SSH key**.
5. Dê um nome (ex: "Meu ThinkPad") e **cole a chave** no campo "Key".
6. Clique em **Add SSH key**.

Como testar se agora deu certo

Volte ao Git Bash e digite novamente o comando do seu print:

Bash

```
ssh -T git@github.com
```

Se aparecer: **"Hi eliandro-avelar! You've successfully authenticated..."**, aí sim está tudo pronto para usar no VS Code!

drwxr-xr-x 1 Lenovo 197121 0 Apr 20 14:58 ./
drwxr-xr-x 1 Lenovo 197121 0 Apr 20 14:58 ../
-rw-r--r-- 1 Lenovo 197121 92 Apr 20 14:58 known_hosts
apareceu essas mensagens no git bash, são as chaves devo usar todas elas

Não, essas não são as suas chaves. O que você está vendo é apenas o conteúdo da pasta `.ssh`, e o arquivo `known_hosts` é apenas um registro de que você tentou se conectar ao GitHub anteriormente (ele guarda o "aperto de mão" inicial que você aceitou com "yes" no print anterior).

Para que o GitHub te reconheça, você precisa de um par de chaves: uma **privada** (que fica no seu PC) e uma **pública** (que você envia para o site).

Como não apareceu nenhum arquivo terminado em `.pub`, você ainda não criou sua chave. Siga estes comandos agora no Git Bash:

1. Gerar a chave

Digite o comando abaixo e aperte **Enter** em todas as perguntas que aparecerem (não digite nada, apenas dê Enter):

Bash

```
ssh-keygen -t ed25519 -C "eliandro_avelar@yahoo.com.br"
```

2. Localizar a chave pública

Após terminar o comando acima, digite:

Bash

```
ls -al ~/.ssh
```

Agora você verá dois arquivos novos: `id_ed25519` (sua chave privada, guarde a sete chaves!) e `id_ed25519.pub` (sua chave pública).

3. Copiar e usar

O arquivo que você deve usar no GitHub é o que termina em `.pub`.
Para ver o conteúdo dele e copiar, digite:

Bash

```
cat ~/.ssh/id_ed25519.pub
```

O que fazer agora?

- Copie todo o código que começar com `ssh-ed25519...`.
- Vá no seu **GitHub -> Settings -> SSH and GPG keys -> New SSH Key**.
- Cole lá e salve.

Depois disso, aquele comando `ssh -T git@github.com` finalmente vai te dar as boas-vindas em vez de dar erro!

$ ssh-keygen -t ed25519 -C "eliandro_avelar@yahoo.com.br"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/Lenovo/.ssh/id_ed25519):…

Agora é só apertar **Enter** até o final!

Como você está no meio do processo, siga estes 3 passos rápidos:

1. **Aperte Enter agora:** Quando ele pergunta em qual arquivo salvar (`Enter file in which to save the key`), apenas aperte **Enter** para aceitar o local padrão.
2. **Aperte Enter na senha (Passphrase):** Ele vai perguntar por uma senha (`Enter passphrase`). Se você não quiser ter que digitar uma senha toda vez que fizer um `push`, deixe em branco e aperte **Enter**.

3. **Aperte Enter de novo:** Confirme a senha vazia apertando **Enter** novamente.

O que vai acontecer?

O terminal vai mostrar um desenho estranho feito de caracteres (um "randomart image"). Isso significa que sua chave foi criada com sucesso!

Próximo passo (O mais importante):

Assim que o desenho aparecer e o terminal liberar para você digitar de novo, use este comando para ver sua chave pública:

Bash

```
cat ~/.ssh/id_ed25519.pub
```

Copie todo o texto que ele exibir (começando por `ssh-ed25519` até o seu e-mail) e cole lá no seu perfil do GitHub (em **Settings > SSH keys**).

Conseguiu gerar o desenho no terminal?

Visualização da imagem enviada

e agora

Análise

Perfeito! Você gerou a chave com sucesso (o desenho do "quadrado" confirma isso).

Agora falta o passo final para o GitHub te reconhecer. Siga exatamente estes comandos:

1. Mostre sua chave pública

Digite este comando no Git Bash para exibir o código da chave:

Bash

```
cat ~/.ssh/id_ed25519.pub
```

2. Copie o resultado

Vai aparecer uma linha comprida começando com `ssh-ed25519` e terminando com seu e-mail. **Selecione tudo isso e copie** (clique com o botão direito e selecione *Copy*).

3. Cole no GitHub

1. Abra o seu **GitHub** no navegador.
2. Clique na sua **foto de perfil** (topo à direita) -> **Settings**.
3. No menu à esquerda, procure por **SSH and GPG keys**.
4. Clique no botão verde **New SSH key**.
5. Em **Title**, coloque um nome (ex: `Notebook Lenovo`).
6. Em **Key**, cole o código que você copiou do terminal.
7. Clique em **Add SSH key**.

---

4. O Teste Final

Depois de salvar no site, volte ao Git Bash e digite:

Bash

```
ssh -T git@github.com
```

Se ele responder: **"Hi eliandro-avelar! You've successfully authenticated..."**, parabéns! Seu VS Code e seu Git estarão totalmente prontos para trabalhar.

Conseguiu copiar o código que o `cat` mostrou?

obrigado!

De nada! Que bom que deu certo. 🚀

Agora que a autenticação está configurada, você pode fazer seus `git push` e `git pull` direto pelo VS Code ou pelo terminal sem erros de permissão.

Se precisar de ajuda com mais algum comando ou com o fluxo de trabalho no Git, é só chamar. Bom código!

qual a importância do git ash estar se comunicando corretamente com o gitHub

Essa comunicação correta é o que permite que o seu trabalho saia do seu computador e vá para a nuvem de forma segura. Sem isso, o Git é apenas uma ferramenta de organização local; com isso, ele se torna uma ferramenta de **colaboração e backup**.

Aqui estão os pontos principais de por que isso é fundamental:

1. Enviar e Receber Código (Push & Pull)

É a ponte que permite você usar os comandos principais:

- `git push` **:** Envia suas alterações para o servidor do GitHub.
- `git pull` **:** Traz as alterações de outras pessoas (ou de outros computadores seus) para a sua máquina.

2. Segurança sem Senhas Chata (SSH)

Como você configurou a **chave SSH**, a comunicação agora é "automática e segura".

- Você não precisa digitar seu usuário e senha toda vez que fizer um envio.

- A chave garante que apenas o seu computador (que tem a chave privada) possa mexer nos seus repositórios, protegendo seu código de acessos não autorizados.

3. Integração com o VS Code

O VS Code usa o Git Bash por baixo dos panos. Quando a comunicação está certa:

- As setas de sincronização do VS Code funcionam.
- Você consegue visualizar conflitos de código e resolvê-los direto na interface gráfica.
- O status do seu projeto (quais arquivos mudaram) fica sempre atualizado com o servidor.

4. Backup e Histórico

Se o seu computador tiver um problema técnico hoje, o seu código está salvo no GitHub. Se a comunicação estiver falhando e você não conseguir dar `push`, seu trabalho corre o risco de ser perdido se algo acontecer com o hardware.

5. Colaboração em Equipe

No mundo profissional, ninguém trabalha sozinho. Para trabalhar em projetos com outras pessoas, você precisa que o seu Git "converse" com o repositório central para que todos vejam a versão mais recente do sistema.

Resumindo: É a diferença entre ter um diário trancado na sua gaveta (Git local) e ter um documento compartilhado na nuvem (GitHub). Agora que você conectou os dois, seu fluxo de trabalho profissional começou de verdade!

Já tentou fazer o seu primeiro **commit** e **push** de um projeto do VS Code para o GitHub?

antes de eu continuar lendo o restante, achei muito interessante esse trecho, mas fiquei com dúvidas, acho melhor tirá-las antes de continuar. Isso significa que eu não preciso fazer esses mesmos procedimentos em minhas outras máquinas, é só logar com meu email qu…

Essa é uma dúvida excelente e muito comum! A resposta curta é: **você precisará repetir esse procedimento em cada nova máquina que for usar.**

Muitas pessoas acham que a conexão é baseada apenas no login do e-mail, mas o SSH funciona como uma **"chave física digital"**.

Por que você precisa repetir o processo?

Imagine que o seu GitHub é um **cofre** e a chave SSH que você criou agora é uma **chave física**.

- Você criou a chave no seu **ThinkPad**.
- Você deu uma cópia (chave pública) para o **GitHub**.
- Agora, o cofre só abre se alguém chegar com a chave que está "espetada" no ThinkPad.

Se você mudar de computador:

1. O novo computador não terá o arquivo da "chave privada" (aquele `id_ed25519` sem o `.pub`).
2. Mesmo que você logue com seu e-mail no VS Code, o computador novo não terá a "identidade digital" para assinar a comunicação segura.

O que você terá que fazer em outras máquinas:

1. **Instalar o Git.**
2. **Configurar seu nome e e-mail** (`git config --global user.name` e `user.email`). Isso serve apenas para "etiquetar" quem fez o código.
3. **Gerar uma nova chave SSH** (o comando `ssh-keygen`).

4. **Adicionar essa nova chave ao GitHub.** * O GitHub permite que você tenha várias chaves cadastradas (uma para o PC de casa, uma para o notebook, uma para o trabalho).

E o VS Code?

O login que você faz no VS Code (clicando no bonequinho no canto inferior esquerdo) serve para sincronizar suas **configurações** (tema, extensões, atalhos), mas ele não transfere suas chaves de segurança de um PC para o outro por questões de proteção.

Dica de ouro: Sempre dê nomes claros para as chaves no GitHub, como "Notebook-Dell" ou "PC-Gamer", para saber qual chave pertence a qual máquina caso você precise remover o acesso de alguma delas um dia.

Ficou clara essa diferença entre "logar com e-mail" e "ter a chave física no PC"?

> tudo bem entendi vou repetir o processo em outras máquinas, mas devo usar a mesma chave gerada pelo git bash no vsCode ou devo gerar uma nova chave no git basho para cada máquina

O ideal e o mais seguro é **gerar uma nova chave em cada máquina**.

Embora seja tecnicamente possível copiar o arquivo da chave privada de um computador para o outro, isso não é recomendado. Aqui estão os motivos para gerar uma chave única por dispositivo:

1. Segurança (O motivo principal)

Se você usa a mesma chave em cinco computadores e perde um deles (ou ele é roubado), você terá que trocar a chave em **todas** as outras máquinas e no GitHub.
Se cada máquina tem sua própria chave, você simplesmente vai no GitHub e deleta apenas a chave daquela máquina perdida. As outras continuam funcionando normalmente.

2. Rastreabilidade

No GitHub, você verá exatamente de qual computador veio cada alteração. Isso ajuda a organizar seu ambiente de trabalho.

3. Praticidade

É muito mais fácil e rápido rodar o comando `ssh-keygen` do que ficar transportando arquivos sensíveis (chaves privadas) via pen drive ou nuvem (o que é perigoso, pois se alguém interceptar sua chave privada, terá acesso ao seu código).

Resumo do que fazer em uma máquina nova:

Quando você sentar em um computador novo, o roteiro é este:

1. Abre o **Git Bash**.
2. Roda o `ssh-keygen -t ed25519 -C "seu_email@yahoo.com.br"` .
3. Copia a chave pública nova (`cat ~/.ssh/id_ed25519.pub`).
4. Cola no GitHub como uma **nova chave** (dando um nome como "PC-Escritorio").

Uma observação importante: O VS Code não tem uma "chave própria". Ele sempre vai tentar usar a chave que está na pasta `~/.ssh` do sistema. Ou seja, uma vez que você configurou o Git Bash, o VS Code "pega carona" nessa configuração automaticamente.

Faz sentido para você manter uma "chave para cada porta" em vez de uma "chave mestra"?